

git commands

- 1) `git --version`: To Check installed git version
- 2) `git init`: To Initialize an empty git repository
- 3) `git status`: To check the status of local repository

4) `git add` command

Usage: To add Untracked Files or Modified Files or deleted files into staging area.

Case 1: Add specific file into staging area.

Syntax: `git add File_Name`

Case 2: Add multiple files into staging area.

Syntax: `git add File1 File2 File3 ...`

Note: File names should be separated by space.

Case 3: Add specific extension files into staging area.

Syntax: `git add pattern`

Example: `git add *.java` (All Java Files)

Case 4: Add All Working area files into staging area.

Syntax: `git add --a`

Case 5: Add all files from current directory into staging area.

Syntax: `git add .` (here is dot symbol)

Case 6: Add Directory into staging area.

Syntax: `git add directory_name`

Case 7: File is added in ".gitignore" but you want to add it into staging area.

Syntax: `git add -f file_name`

5) git rm command

-> Usage of git rm command

- 1) To unstage a file.
- 2) To unstage a directory
- 3) To unstage a file and delete from system.
- 4) To unstage a directory and delete from System

-> options of git rm command

- 1) `--cached` : To Unstage a file
- 2) `-f` : To Unstage and delete from system
- 3) `-r --cached` : Unstage a directory
- 4) `-rf` : Unstage and delete directory from system

-> Different Cases:

Case 1: To unstage File or Files

Syntax: `git rm --cached File1_Name File2_name File3_name`

Case 2: To Unstage a directory

Syntax: `git rm --cached -r directory_name`

Case 3: Unstage files and delete from System

Syntax: `git rm -f File1_Name File2_name File3_name`

Case 4: Unstage a directory and delete

Syntax: `git rm -rf directory_name`

6) git restore command

-> Usage of git restore command

- 1) To Unstage single file or multiple files.
- 2) To discard single file content or multiple file content to previous state.

-> Options of git restore command

1) --staged : To Unstage a file or files

-> Different Cases:

Case 1: Unstage file or Files

Syntax: git restore --staged File1_Name File2_name

If we want to unstage single file then specify single file name. If we want to unstage multiple files then specify multiple file names.

Case 2: Unstage all files at a time

Syntax: git --restore . (here is dot symbol)

Case 3: Restore single file content to previous state.

Syntax: git restore file_name

Case 4: Restore multiple file content to previous state.

Syntax: git restore File1_Name File2_name File3_name

Case 4: Restore all modified file changes to previous state

Syntax: git restore .(here use dot symbol)

Note: When we want to discard the file content and restore the file to previous state, first unstage it, then discard the changes.

Note: git restore command with --staged option, unstage file and git restore command without --staged option discard the file changes and restore the file to previous state.

7) git branch command.

-> Usage of "git branch" command

- 1) To list out local branches
- 2) To list out all branches(local+remote)
- 3) To create a new branch
- 4) To clone a remote branch
- 5) To delete a branch
- 6) To rename a branch.

-> Different Cases

Case 1: To list out local branches

Syntax: git branch

Case 2: To list out all branches

Syntax: git branch -a

Case 3: To Create a new branch

Syntax: git branch branch-name

Case 4: To clone remote branch

Syntax: git branch -b branch-name repository_url

Case 5: To delete single branch at a time

Syntax: git branch -d branch-name

Case 6: To delete multiple branches at a time

Syntax: git branch -d branch-name1 branch-name2 branch-name3

Case 7: To rename current branch

Syntax: `git branch -m new_name`

Case 8: Rename other branch from current branch

(Currently you on `xyz` branch and you want rename other branch)

Syntax: `git branch -m old_name new_name`

8) git checkout command

-> Usage of git checkout command

- 1) To switch from one branch to another branch
- 2) Create a branch and switch to that branch
- 3) To checkout remote branch

-> Different Cases

Case 1: To switch from current to another branch

Syntax: `git checkout branch-name`

Case 2: Create a branch and switch to that branch

Syntax: `git checkout -b branch-name`

Case 3: Checkout remote branch

Syntax: `git checkout -b branch-name origin/branch-name`

8) git switch command

-> Usage of git switch command

- 1) To switch from one branch to another branch
- 2) Create a branch and switch to that branch

-> Different Cases

Case 1: To switch from current to another branch

Syntax: `git switch branch-name`

Case 2: Create a branch and switch to that branch

Syntax: `git switch -c branch-name`

9) git diff command

-> Usage:

- 1) comparing working area file changes with last commit.
- 2) comparing staging area file changes with last commit.
- 3) comparing two commits.
- 4) comparing current branch changes with another branch
- 5) comparing current changes with remote branch

-> Use cases

Case 1: comparing working area file changes with last commit

Syntax: git diff

Case 2: comparing staging area file changes with last commit.

Syntax: git diff --staged

Case 3: comparing two commits

Syntax: git diff commit_id_1 commit_id_2

Case 4: comparing current branch changes with another branch

Syntax: git diff another-branch-name

Case 5: comparing current changes with remote branch

Syntax: git diff remote_branch_name

Example: git diff origin/feature-01

10) git log command

-> Usage: To view the commit history of repository

-> Syntax: git log <option> (Option is optional)

-> options:

- 1) --oneline: Display each commit in one line
- 2) --graph : Display graphical representation of commit history.
- 3) --stat: Display summary of changes introduced in each commit
(Display number of lines added, deleted and modified)
- 4) -p or -patch: Display actual changes made in each commit
- 5) -n : To limit number commits displayed.
- 6) --since and --until: Display commit history for specific time duration.

Example: git log --since "2 week ago"

Example: git log --since "2 day ago"

Example: git log --until "2024-08-23" (Date Format is YYYY-MM-DD)

Note: we can use multiple options in single command

Example: git log --oneline -n 4

Note: Order of option is not important

11) git config command

-> Usage: To customize the git Behavior

-> options of git config

- 1) --list
- 2) --system
- 3) --local
- 4) --global

-> Different Use Cases

Case 1: List out local level configuration

Syntax: `git config --list --local`

Case 2: List out global level configuration

Syntax: `git config --list --global`

Case 3: List out system level configuration

Syntax: `git config --list --system`

Case 4: List out system+global level configuration

Syntax: `git config --list`

Note: To view configuration setting we need to use --list option and level (scope) of configuration

Case 5: Configure user name and email(local level)

Syntax: `git config --local user.name "user_name"`
`git config --local user.email "email"`

Case 6: Configure user name and email(global level)

Syntax: `git config --global user.name "user_name"`
`git config --global user.email "email"`

Case 7: Configure user name and email(system level)

Syntax: `git config --system user.name "user_name"`
`git config --system user.email "email"`

Case 8: Configure default(initial) branch name as "main"

Syntax: `git config --global init.defaultBranch "main"`